OWLBOX

Verkabelung

Vollständige Hardware-Referenz:
Pinbelegung, DSI-Display, HiFiBerry, Netzwerk-Fallback.

Hardware-Aufbau Raspberry Pi 5
Schnelleinstieg: OwlBox-Schnellstart.pdf

Inhalt

Inhalt 2

1. Zielhardware 4

Anschluss: 5″ Waveshare DSI Touch Display 4

GPIO-Belegung im Überblick 5

2. Komponentenübersicht 7

Zentraleinheit 7

Audio (Kapitel 3) 7

Display (Kapitel 1.1 und 10) 7

RFID (Kapitel 5) 8

Bedienelemente (Kapitel 6 und 7) 8

Verkabelung & Adapter-Platine (Kapitel 3.2) 8

Werkzeug (nicht Teil der Stückliste, aber sinnvoll) 9

3. HiFiBerry Amp2 10

Wichtig: vc4-kms-v3d braucht ,noaudio 10

Lautsprecher anschließen 11

3.1 Aktive Kühlung (Pi 5) 12

3.2 Nicht mehr direkt aufgesteckt: Anschluss über Adapter-Platine 12

4. AirPlay (optional, shairport-sync) 14

5. RC522 RFID-Leser (Hardware-SPI0) 15

6. Taster (vor/zurück) 16

7. Dreh-Encoder mit Taster (KY-040) 17

Zweiter Dreh-Encoder für Helligkeit (KY-040) 17

8. Display-Hintergrundbeleuchtung 19

9. Fallback-Hotspot (WLAN-Recovery) 20

10. DSI-Display: kein separater Treiber-Installer nötig 21

Drehung/Ausrichtung: bewusst nicht konfiguriert 21

11. Kiosk-Autostart (Chromium fullscreen, ohne Desktop-Umgebung) 22

12. Konfigurationsdatei (config.yaml) 24

1. Zielhardware

- Raspberry Pi 5 (4GB) mit aktiver Kühlung (siehe Kapitel 3). Ersetzt das früher hier dokumentierte Pi 3B+; dessen vollständig verifiziertes Setup ist noch in der Git-Historie dieser Datei zu finden. Noch NICHT an echter Hardware verifiziert.
- HiFiBerry Amp2 (I2S-Verstärker-HAT, TAS5756M-Chip) - sitzt wegen des Kühlkörpers NICHT mehr direkt gestapelt auf dem 40-Pin-Header, sondern hängt über eine eigene Adapter-Platine dran, per Jumperkabel wie RC522/Taster/Encoder auch (siehe Kapitel 3). Noch NICHT an echter Hardware verifiziert.
- RC522 RFID-Modul (SPI, 13,56 MHz)
- Waveshare 5″ DSI Capacitive Touch Display (Modell 5-DSI-TOUCH-A, 720×1280, DSI-Flachbandkabel für Bild und Touch, plus 4 Jumperkabel für Strom/I2C, siehe unten). Ersetzt das früher hier dokumentierte offizielle Raspberry-Pi-7″-Touch-Display (800×480); dessen Anleitung ist noch in der Git-Historie dieser Datei zu finden, falls je wieder gebraucht. Noch NICHT an echter Hardware verifiziert, siehe Kapitel 1.1.
- 2 Taster (vor/zurück)
- 1 Dreh-Encoder mit Druckschalter (Lautstärke/Play-Pause)
- 1 weiterer Dreh-Encoder mit Druckschalter (Helligkeit, Taster schaltet den Nachtmodus, s.u.)

Alle Pin-Angaben sind BCM-Nummerierung und entsprechen den Standardwerten in config/config.example.yaml. Wer andere Pins verdrahtet, passt einfach die gpio:/rfid:-Sektion in config.yaml an.

Hinweis: Referenzgrafik zusätzlich zu dieser PDF im Projekt: docs/owlbox-gpio-pinout.svg (kompletter 40-Pin-Header, direkte Jumperkabel wie in dieser PDF). Für eine eigene, separat verdrahtete Adapter-Platine statt direkter Jumperkabel (optional, nicht Teil des Standardaufbaus): docs/owlbox-wiring-diagram.svg (Gesamtaufbau), docs/owlbox-adapter-layout.svg (Platinenlayout-Vorschlag) und docs/hat-wiring.html (kompletter Schaltplan, im Browser öffnen) - alle drei aktuell für RC522 auf Hardware-SPI0/CE0; die Display-Zeilen zeigen nur die generischen Strom-/I2C-Adern, nicht die produktspezifische Overlay-/Auflösungs-Konfiguration (siehe unten).

Anschluss: 5″ Waveshare DSI Touch Display

Wichtig: Dieser Abschnitt ist noch NICHT an echter Hardware verifiziert - anders als der Rest dieser PDF. Das Display (Waveshare 5-DSI-TOUCH-A, 720×1280, kapazitiver Touch, Aluminiumgehäuse) löst das bisherige offizielle 7″-Touch-Display (800×480) ab; dessen vollständig verifizierte Anleitung bleibt in der Git-Historie dieser Datei erhalten.

Wie beim bisherigen Display sitzt es **nicht** auf dem 40-Pin-Header - Bild und Touch laufen über das DSI-Flachbandkabel (eigener Steckplatz auf dem Pi, neben den HDMI-Buchsen), keine Steckplatz-Kollision mit dem HiFiBerry (der auf diesem Aufbau ohnehin nicht mehr direkt auf dem 40-Pin-Header sitzt, sondern über eine Adapter-Platine läuft, siehe Kapitel 3).

Die Adapter-/Power-Platine des Displays braucht vermutlich weiterhin 4 Jumper-/Dupont-Kabel zum Pi (Strom + I2C für den Touch-Controller), analog zum bisherigen Display:

Display-Adapterplatine	Pi-Pin (BCM)	Zweck
5V	Pin 2 oder Pin 4	Stromversorgung
GND	Pin 6 (oder jeder andere GND-Pin)	Masse
SDA	Pin 3 (GPIO2)	I2C-Datenleitung (Touch-Controller)
SCL	Pin 5 (GPIO3)	I2C-Taktleitung (Touch-Controller)

Wichtig: Bitte gegen das Waveshare-eigene Handbuch/Wiki prüfen, sobald das Display da ist - diese Tabelle ist vom bisherigen Display übernommen (gleiches Funktionsprinzip: DSI + separate Stromversorgung + I2C-Touch), aber nicht produktspezifisch bestätigt.

Hinweis: Kein Konflikt mit dem HiFiBerry, obwohl GPIO2/3 dieselben Pins sind, die er für seine eigene I2C-Steuerung nutzt: I2C ist ein echter Mehrgeräte-Bus, mehrere Chips teilen sich Takt-/Datenleitung problemlos, solange sie unterschiedliche Adressen haben - vorausgesetzt, der Touch-Controller dieses Displays sitzt tatsächlich auch auf I2C.

(Hinweis aus dem früheren, direkt gestapelten Aufbau, hier nur noch der Vollständigkeit halber: säße der HiFiBerry doch einmal wieder direkt auf dem 40-Pin-Header und die Pins dadurch von oben nicht mehr mit Dupont-Kabeln erreichbar, würde ein GPIO-Stacking-Header [Extra-Höhe, mit durchgeführten Pins] zwischen Pi und HiFiBerry das lösen. Auf dem aktuellen Aufbau nicht relevant, da der HiFiBerry ohnehin über die Adapter-Platine läuft, s. Kapitel 3.)

Wichtig: Overlay laut Waveshare-Wiki (waveshare.com/wiki/5-DSI-TOUCH-A - nicht direkt erreichbar geprüft, nur über Suchergebnisse, bitte gegenlesen): dtparam=i2c_arm=on plus dtoverlay=vc4-kms-dsi-waveshare-panel-v2,5_0_inch_a. NICHT der bisherige dtoverlay=vc4-kms-dsi-7inch (spezifisch fürs alte Display) und NICHT das ähnlich klingende vc4-kms-dsi-waveshare-panel,5_0_inch ohne "-v2"/"_a" (andere Waveshare-5″-Modelle). install.sh trägt den Overlay automatisch ein. Rotation/Ausrichtung bewusst nicht konfiguriert - wird laut Auftraggeber beim physischen Einbau gelöst, nicht per Software; Details siehe Kapitel 10.

GPIO-Belegung im Überblick

Funktion	BCM-Pin	Genutzt von
I2S BCLK	18	HiFiBerry
I2S LRCLK	19	HiFiBerry
I2S PCM_DIN (Eingang am Pi)	20	HiFiBerry (vom Amp2 ungenutzt)
I2S PCM_DOUT (Audiodaten zum Amp)	21	HiFiBerry

Funktion	BCM-Pin	Genutzt von
I2C SDA	2	HiFiBerry (Amp-Steuerung) und Display-Touch-Controller - gemeinsam am selben I2C-Bus, kein Konflikt (unterschiedliche Adressen), s.o.
I2C SCL	3	HiFiBerry (Amp-Steuerung) und Display-Touch-Controller, s.o.
SPI0 SCLK/MOSI/MISO/CE0	11 / 10 / 9 / 8	RC522 (Hardware-SPI, s. Kapitel 5)
SPI0 CE1	7	frei (nicht genutzt - der RC522 braucht nur CE0)
RC522 RST	26	RC522 (rfid.reset_pin)
Taster Weiter	5	Taster
Taster Zurück	6	Taster
Encoder CLK	1	Lautstärke-Encoder (17 ist inzwischen wieder belegt, s.u.)
Encoder DT	27	Lautstärke-Encoder
Encoder SW	22	Lautstärke-Encoder
Display-Backlight	-	läuft über Sysfs, kein GPIO mehr - dimmt zuverlässig über den Encoder unten, s. Kapitel 8
Helligkeits-Encoder CLK	23	Helligkeits-Encoder
Helligkeits-Encoder DT	12	Helligkeits-Encoder
Helligkeits-Encoder SW	17	Helligkeits-Encoder - Nachtmodus-Umschalter (gpio.brightness_encoder_switch), s.u.

2. Komponentenübersicht

Bevor es an die einzelnen Pin-Tabellen geht: welches Bauteil und welches Kabel pro Baugruppe gebraucht wird. Angaben zu Typ/Spezifikation/Menge, bewusst ohne Preise oder konkrete Händlerlinks - die schwanken zu schnell, um sie in einem Dokument zu pflegen. Bei allen mit „Noch nicht an echter Hardware verifiziert“ markierten Positionen gilt dieselbe Einschränkung wie im Rest dieser PDF: vor dem Kauf gegen das jeweilige Datenblatt prüfen.

Zentraleinheit

Bauteil	Spezifikation	Menge
Raspberry Pi 5 (4GB)	Quad-Core Cortex-A76, 40-Pin-GPIO-Header, DSI- und CSI-Anschluss. Ersetzt das früher verbaute Pi 3B+.	1×
GeeekPi Low-Profile Plus CPU Cooler	Aluminium-Kühlkörper mit Lüfter, steckt auf den eigenen 4-Pin-JST-Lüfteranschluss des Pi 5 (kein GPIO, keine config.txt-Zeile nötig, s. Kapitel 3.1).	1×
USB-C-PD-Netzteil	5V/5A (27W, offizielles Raspberry-Pi-Netzteil empfohlen) - ein schwächeres 5V/2,5-3A-Netzteil reicht bei Amp2 unter Last plus Lüfter nicht sicher aus.	1×
microSD-Karte	Mind. 16GB, empfohlen 32GB+, Class 10 / A2 für flüssiges Booten und die Hörspiel-/Spiele-Bibliothek.	1×

Audio (Kapitel 3)

Bauteil	Spezifikation	Menge
HiFiBerry Amp2	I2S-Verstärker-HAT, TAS5756M-Chip, Class-D, 2 Kanäle. Sitzt wegen des Kühlkörpers nicht mehr direkt auf dem 40-Pin-Header, sondern hängt per Jumperkabel an der Adapter-Platine (s. Kapitel 3.2).	1×
Passivlautsprecher	Impedanz/Belastbarkeit gegen das Amp2-Datenblatt prüfen (typ. 4-8Ω).	2×
Lautsprecherkabel, 2-adrig	Querschnitt ≥ 0,75mm² / AWG18 (reicht für 15W/4Ω); bei Kabelwegen über 3-5m eher 1,0-1,5mm² nehmen. Für die Federklemmen des Amp2, kein Cinch/Klinke.	nach Bedarf

Display (Kapitel 1.1 und 10)

Wichtig: Noch NICHT an echter Hardware verifiziert.

Bauteil	Spezifikation	Menge
Waveshare 5″ DSI Capacitive Touch Display	Modell 5-DSI-TOUCH-A, 720×1280, kapazitiver Touch, Aluminiumgehäuse. Bild+Touch über eigenes DSI-Flachbandkabel (Lieferumfang), zusätzlich 4 Jumperkabel für Strom/I2C.	1×
Jumperkabel, Dupont female-female	Für Display-Strom (5V, GND) und I2C-Touch (SDA, SCL) zwischen Display-Adapterplatine und Pi-GPIO-Header.	4×

RFID (Kapitel 5)

Bauteil	Spezifikation	Menge
RC522-Modul	SPI-RFID-Leser, 13,56 MHz, läuft am Pi über Hardware-SPI0/CE0.	1×
Jumperkabel, Dupont female-female	Für SPI (SCLK/MOSI/MISO/CE0), RST und 3,3V/GND - VCC ausdrücklich an 3,3V, nicht 5V.	7×
RFID-Chips/Tags	13,56 MHz, Mifare-kompatibel (Karte, Sticker oder Schlüsselanhänger) - je ein Chip pro Hörspiel/Funktion, plus optional je einer pro Nutzer-Login.	nach Bedarf

Bedienelemente (Kapitel 6 und 7)

Bauteil	Spezifikation	Menge
Taster (Cherry MX oder kompatibel)	3-Pin-Bauform, nur 2 Metallpins aktiv genutzt - vor/zurück.	2×
KY-040 Dreh-Encoder-Modul, mit Druckschalter	Lautstärke/Play-Pause; langer Druck fährt den Pi sicher herunter.	1×
KY-040 Dreh-Encoder-Modul, mit Druckschalter	Helligkeit; Druckschalter schaltet den Nachtmodus um.	1×

Verkabelung & Adapter-Platine (Kapitel 3.2)

Bauteil	Spezifikation	Menge
Lochraster-Platine	Trägt die 40-Pin-Buchsenleiste für den Amp2 plus die Jumperkabel-Anschlüsse für RC522/Taster/Encoder - eigenverdrahtet, kein fertiges Produkt.	1×
40-Pin-Buchsenleiste (2×20, 2,54mm), zum Auflöten	Nimmt den Amp2 als vollständiges 40-Pin-HAT auf - anders als RC522/Taster/Encoder lässt sich der Amp2 nicht einzeln per Jumperkabel verdrahten.	1×
Jumperkabel, Dupont male-female	Verbindet die 40-Pin-Buchsenleiste der Adapter-Platine mit dem 40-Pin-Header des Pi.	40×

Bauteil	Spezifikation	Menge
Litze, AWG20 oder dicker (für 5V/GND des Amp2)	Der Amp2 zieht seine komplette Lautsprecher-Ausgangsleistung direkt aus der 5V-Schiene (Class-D) - bei Zimmerlautstärke durchaus über 1A. Dünne Standard-Jumperkabel sind dafür nicht ausgelegt.	kurz, je 2×

Hinweis: Nach dem Zusammenbau prüfen: vcgencmd get_throttled sollte 0x0 zeigen (keine Unterspannung) - bei Verzerren/Aussetzern unter Last zuerst hier ansetzen.

Werkzeug (nicht Teil der Stückliste, aber sinnvoll)

- Lötkolben + Lötzinn (für die 40-Pin-Buchsenleiste auf der Adapter-Platine)
- Abisolierzange (für die Lautsprecher- und Netzteil-Adern)
- Multimeter (Polaritäts-/Kurzschlussprüfung vor dem ersten Einschalten)
- microSD-Kartenleser (zum Flashen des OS-Images, falls nicht schon vorhanden)

3. HiFiBerry Amp2

Der HiFiBerry belegt die I2S-Pins (BCM 18/19/20/21) sowie I2C (BCM 2/3) zur Verstärkersteuerung. In /boot/firmware/config.txt (bzw. /boot/config.txt auf älteren Images):

```
dtparam=audio=off
dtoverlay=hifiberry-dacplus
```

Wichtig: Auf dem Pi 5 stattdessen dtoverlay=hifiberry-dacplus-std - ein kernelseitiger Probe-Fix, spezifisch für die BCM2712-SoC/RP1-Kombination des Pi 5 (laut HiFiBerry und dem raspberrypi/linux-Issue-Tracker). Noch NICHT an echter Pi-5-Hardware verifiziert. install.sh wählt das automatisch anhand des erkannten Boards (/proc/device-tree/model).

Hinweis: An echter Hardware bestätigt: Der Amp2 hat einen TAS5756M-Chip - dieselbe PCM512x-Chipfamilie wie die DAC+ Pro, ein komplett anderer Chip als der TAS5713 des älteren Amp/Amp+. dtoverlay=hifiberry-amp ist speziell für den TAS5713 und funktioniert mit dem Amp2 nicht: der Kernel spricht dann die falsche I2C-Adresse an, aplay -l zeigt „no soundcards found“ - äußert sich als Lautstärke, die sich nie ändert (bleibt bei 0). Zum Nachprüfen, welcher Chip verbaut ist: i2cdetect -y 1 - Adresse 0x4d antwortet beim TAS5756M/Amp2 (hifiberry-dacplus), Adresse 0x1b beim TAS5713 vom Amp/Amp+ (hifiberry-amp).

Nach einer Overlay-Änderung neu starten - dabei verschiebt sich meist auch die Kartennummer.

Danach mit `aplay -l` die Kartennummer der HiFiBerry ermitteln (z.B. „card 2: ...“) und mit `amixer -c <Kartennummer> scontrols` den Mixer-Namen prüfen - beides in config.yaml eintragen: audio.alsa_device ("hw:<Kartennummer>,0"), audio.mixer_control (Amp/Amp2 nutzen meist „Digital“, manche Boards „PCM“ oder „Master“) **und audio.mixer_card** (nur die Kartennummer, ohne hw:/,0).

Wichtig: Alle drei müssen zur selben Karte passen - install.sh trägt sie bei der automatischen Erkennung mittlerweile alle drei ein, aber wer das von Hand einträgt, vergisst leicht mixer_card: bleibt die auf ihrem Standardwert "0" stehen während die HiFiBerry tatsächlich auf einer anderen Kartennummer läuft, zielt jede Lautstärkeabfrage/-änderung ins Leere - äußert sich als „eingestellte Lautstärke wird nie gespeichert, zeigt immer 0“.

Wichtig: vc4-kms-v3d braucht ‚noaudio

Der vc4-kms-v3d-Grafiktreiber-Overlay registriert standardmäßig zusätzlich eine eigene HDMI-Audio-ALSA-Karte (taucht in aplay -l als „card N: vc4hdmi“ auf), auch wenn HDMI-Audio in diesem Projekt nie genutzt wird (Anzeige läuft über DSI, Ton ausschließlich über den HiFiBerry).

Wichtig: An echter Hardware bestätigt, Ursache eines tagelangen „Wiedergabe knackt/fragmentiert trotz digital korrektem Signalweg“-Rätsels: Diese HDMI-Audio-Registrierung kollidiert offenbar mit dem I2S-Pfad des HiFiBerry (beide laufen letztlich über denselben VC4-I2S/Audio-Hardwareblock) - äußert sich als digital sauber ankommende, aber physisch knacksende/fragmentierte Wiedergabe, dazu wiederkehrende pcm512x-I2C-Fehler im Kernel-Log. Keins der naheliegenden Gegenmittel (Auto Mute an/aus, Bluetooth deaktivieren, Runtime-Power-Management-Sysfs-Override, selbst ein komplett frisches SD-Karten-Image) behebt das, weil keins davon die eigentliche Ursache anfasst.

Der Fix: ,noaudio an den Overlay anhängen, damit vc4-kms-v3d sich aus der Audio-Seite dieses gemeinsam genutzten Hardwareblocks komplett heraushält:

```
dtoverlay=vc4-kms-v3d,noaudio
```

install.sh trägt das automatisch so ein.

Wichtig: Zweite Falle, an echter Hardware bestätigt: Der Fix wirkt nur, wenn er die EINZIGE dtoverlay=vc4-kms-v3d-Zeile in der Datei ist. Ein frisches Raspberry Pi OS Bookworm-Image bringt in config.txt bereits eine eigene, unkommentierte dtoverlay=vc4-kms-v3d-Zeile mit (ohne ,noaudio). Anders als dtparam=-Zeilen sind dtoverlay=-Zeilen KEINE Key/Value-Overrides - jede einzelne wendet den Overlay unabhängig an. Stehen beide Zeilen in der Datei, registriert die erste trotzdem ihre eigene vc4hdmi-Karte, und das Knacksen bleibt bestehen. Kontrolle: grep -n dtoverlay=vc4-kms-v3d config.txt sollte genau einen Treffer zeigen (den mit ,noaudio); aplay -l sollte keine vc4hdmi-Karte mehr auflisten. install.sh passt die vorbestehende Zeile jetzt automatisch direkt an Ort und Stelle an (statt sie zu löschen und eine eigene Kopie ans Dateiende anzuhängen) - auf einer schon länger laufenden Installation reicht dafür ein erneutes sudo owlbox-install plus Neustart. Wichtig: dafür wirklich owlbox-install verwenden (ein stabiler Befehl, den das Skript bei seinem ersten erfolgreichen Durchlauf selbst unter /usr/local/bin anlegt), nicht cd owlbox && sudo ./scripts/install.sh - cd owlbox von innerhalb eines bereits ausgecheckten Repos landet nicht im Repo-Root, sondern eine Ebene zu tief im gleichnamigen Python-Paket-Unterordner, und ./scripts/install.sh meldet dann nur „command not found“, ohne dass irgendetwas vom Skript tatsächlich läuft.

Wichtig: Dritte Falle, ebenfalls an echter Hardware bestätigt: dtparam=audio=on muss aus demselben Grund verschwinden, nicht nur auskommentiert oder von einem späteren dtparam=audio=off "überschrieben" werden. Ein frisches Bookworm-Image bringt standardmäßig eine eigene, unkommentierte dtparam=audio=on-Zeile mit. Die naheliegende Annahme - dtparam=-Zeilen seien Key/Value-Overrides, bei denen die letzte Zeile gewinnt, also würde install.sh's eigenes, weiter unten stehendes dtparam=audio=off automatisch siegen - hat sich an echter Hardware NICHT zuverlässig bestätigt: die onboard „bcm2835 Headphones“-ALSA-Karte tauchte trotz korrekt zuletzt stehendem dtparam=audio=off über mehrere Neustarts hinweg immer wieder in aplay -l auf. install.sh kommentiert seit dieser Erkenntnis die vorbestehende dtparam=audio=on-Zeile automatisch direkt an Ort und Stelle aus (#dtparam=audio=on), statt sich auf Override-Semantik zu verlassen - derselbe sudo owlbox-install plus Neustart wie oben behebt beides in einem Rutsch.

Lautsprecher anschließen

Der HiFiBerry Amp2 hat dafür keine Stecker (kein Cinch/Klinke), sondern zwei 2-polige Federklemmen direkt auf der Platine (eine pro Kanal, jeweils +/-). Angeschlossen wird ganz normales 2-adriges Lautsprecherkabel:

- Querschnitt ≥ 0,75 mm² (AWG 18) reicht für 15W/4Ω locker; bei längeren Kabelwegen (> 3-5m) eher 1,0-1,5 mm² nehmen.
- Enden abisolieren (~10mm); bei feindrähtiger Litze verzinnen oder Aderendhülsen verwenden, damit die Federklemme sauber greift.
- Polarität an beiden Lautsprechern konsistent anschließen (+ zu + und Minus zu Minus) - sonst laufen sie gegenphasig und Bass/Stereo-Ortung leiden.
- Am Lautsprecher selbst hängt der Anschluss vom jeweiligen Modell ab (blanker Draht, Flachsteckhülsen/Bananas oder Lötfahnen).

3.1 Aktive Kühlung (Pi 5)

Wichtig: Dieser komplette Abschnitt ist noch NICHT an echter Hardware verifiziert - die Software-Anpassungen sind vorbereitet, aber dieses Projekt lief zum Zeitpunkt des Schreibens noch auf einem Pi 3B+.

Verbaut: GeeekPi Low-Profile Plus CPU Cooler (Aluminium-Kühlkörper mit Lüfter, für Pi 5 4GB/8GB/16GB). Steckt wie der offizielle Raspberry-Pi-„Active Cooler“ auf den eigenen 4-Pin-JST-Lüfteranschluss des Pi 5 (rechts oben, zwischen 40-Pin-Header und den USB-2-Ports) - **kein GPIO-Pin, keine config.txt-Zeile nötig**. Die Drehzahl regelt die Pi-5-Firmware selbst temperaturabhängig (Stufen bei ca. 60°C/67,5°C/75°C), unabhängig vom Betriebssystem. Damit ist die Kühlung für dieses Projekt reine Mechanik - sie taucht in keiner GPIO-Tabelle auf und braucht keine eigene config.yaml-Einstellung.

Hinweis: Nur falls stattdessen doch einmal ein anderer, GPIO-verdrahteter Lüfter verbaut wird (nicht der hier tatsächlich verbaute, nur zur Einordnung): das bräuchte einen zusätzlichen freien BCM-Pin plus einen eigenen Fan-Overlay/eine eigene Steuerlogik - für den GeeekPi-Kühler oben nicht relevant, der läuft komplett über den festen 4-Pin-Anschluss.

Wichtig: Stromversorgung: Der Pi 5 empfiehlt offiziell ein 5V/5A-USB-C-PD-Netzteil (27W) - mit HiFiBerry Amp2 (kann bei Zimmerlautstärke durchaus über 1A aus der 5V-Schiene ziehen) und aktivem Lüfter zusammen an einem schwächeren Netzteil (z.B. die alten 5V/2,5-3A-Netzteile vom Pi-3B+-Aufbau) drohen Unterspannungswarnungen/-drosselung. Für diesen Aufbau (Amp2 unter Last plus Lüfter) das offizielle 27W-Netzteil verwenden.

3.2 Nicht mehr direkt aufgesteckt: Anschluss über Adapter-Platine

Wichtig: Dieser Abschnitt ist noch NICHT an echter Hardware verifiziert.

Wegen des Kühlkörpers auf dem Pi 5 sitzt der Amp2 nicht mehr direkt gestapelt auf dem 40-Pin-Header, sondern hängt über Jumperkabel an einer eigenen, separat verdrahteten Adapter-Platine - demselben Eigenbau-Muster (Lochraster + Jumperkabel), das dieses Projekt für RC522/Taster/Encoder schon einsetzt (docs/hat-wiring.html, dort aber noch nicht auf diesen

Aufbau aktualisiert). Der Amp2 selbst bleibt unverändert - es ändert sich nur, wie seine Pins den Pi erreichen.

HiFiBerry Amp2	Pi-Pin (BCM)	Zweck
BCLK	GPIO18	I2S-Bit-Clock
LRCLK/WS	GPIO19	I2S-Wortauswahl (links/rechts)
DATA	GPIO21 (PCM_DOUT)	I2S-Audiodaten vom Pi zum Verstärker - ohne diese Ader kein Ton
-	GPIO20 (PCM_DIN)	I2S-Eingang am Pi (vom Amp2 ungenutzt für reine Wiedergabe, trotzdem verbinden)
SDA	GPIO2	I2C-Datenleitung (Verstärkersteuerung)
SCL	GPIO3	I2C-Taktleitung (Verstärkersteuerung)

HiFiBerry Amp2	Pi-Pin (physisch)	Zweck
5V	Pin 2 UND Pin 4	Versorgung des kompletten Verstärkers inkl. Lautsprecherausgang
GND	mind. 1-2 GND-Pins (z.B. 6, 9, 14)	Masse

Wichtig: Wichtig, unabhängig von echter Hardware ableitbar (Elektrotechnik, nicht projektspezifisch getestet): Anders als bei RC522/Tastern/Encodern, die nur Milliampere-Signalpegel führen, zieht der Amp2 seine komplette Lautsprecher-Ausgangsleistung direkt aus der 5V-Schiene (Class-D-Verstärker) - bei Zimmerlautstärke können das ohne Weiteres über 1A sein, kurzzeitig bei Bässen/hoher Lautstärke auch mehr. Für diese eine Verbindung NICHT dieselben dünnen Jumper-/Dupont-Kabel wie für RC522/Taster/Encoder verwenden (typischerweise nur für < 1A ausgelegt): beide 5V-Pins UND mehrere GND-Pins parallel nutzen, wenn möglich kurze, dickere Leitungen (z.B. AWG 20 oder dicker) für genau diese beiden Adern. Nach dem Zusammenbau prüfen: vcgencmd get_throttled sollte 0x0 zeigen (keine Unterspannung); bei hörbarem Verzerren/Aussetzern unter Last zuerst hier ansetzen. Kein 3.3V-Pin nötig - der Amp2 hat kein ID_SD/ID_SC-EEPROM (der Overlay wird manuell eingetragen, s.o.). Die genaue Stromaufnahme steht im Datenblatt des Amp2 (HiFiBerry-eigene Seite) - vor dem endgültigen Verkabeln dort noch einmal gegenprüfen.

4. AirPlay (optional, shairport-sync)

Kein zusätzliches Kabel, kein zusätzlicher Chip - AirPlay läuft komplett über WLAN und dieselbe HiFiBerry-Ausgabe, die OwlBox ohnehin schon nutzt. Ein optionales Software-Add-on: sudo ./scripts/install.sh airplay installiert shairport-sync (github.com/mikebrady/shairport-sync), den etablierten Open-Source-AirPlay-Empfänger für Linux, als eigenen systemd-Dienst - bewusst NICHT Teil von sudo ./scripts/install.sh (bzw. "all") ohne Argument, da es kein Kernbestandteil der Box ist.

Hinweis: Zusammenspiel mit der eigentlichen Wiedergabe: mpv (für Geschichten) und shairport-sync (für AirPlay) würden sich sonst dasselbe ALSA-Gerät streitig machen bzw. beide gleichzeitig hörbar übereinanderlaufen. Gelöst über automatisches Ducking statt gemeinsamer ALSA-Nutzung: shairport-sync ruft über seine sessioncontrol-Hooks (run_this_before_play_begins/run_this_after_play_ends in /etc/shairport-sync.conf, vom Installer geschrieben) zwei kleine Skripte auf (scripts/airplay-session-start.sh/-end.sh, per curl gegen 127.0.0.1:5000), die Engine.airplay_session_started()/_ended() antriggern: läuft gerade eine Geschichte, wird sie pausiert (Position gespeichert wie gewohnt); nur falls der Start-Hook tatsächlich pausiert hat, wird beim Session-Ende wieder fortgesetzt. Eine Geschichte, die schon vorher aus einem anderen Grund pausiert war, bleibt pausiert.

Auf dem Kiosk-Display erscheint währenddessen unten rechts ein kleines „ AirPlay“-Abzeichen (owlbox/web/static/js/player.js, gespeist aus state.airplay.active), damit auf einen Blick klar ist, warum eine Geschichte gerade pausiert wirkt. Die beiden /api/airplay/session-start/-end-Endpunkte (owlbox/web/api.py) sind bewusst NICHT @admin_required - ein headless laufendes Systemd-Skript kann keinen Login-Flow durchlaufen - stattdessen auf lokale Aufrufe beschränkt (request.remote_addr gegen 127.0.0.1/::1 geprüft, nicht spoofbar wie ein Header).

AirPlay-Version: das Debian/Raspberry-Pi-OS-Paket unterstützt AirPlay 1 (von praktisch jeder AirPlay-Quell-App weiterhin akzeptiert, nur ohne AirPlay 2s Mehrraum-/„Gerade läuft“-Zusatzfunktionen). AirPlay 2 bräuchte einen Build aus dem Quellcode mit zusätzlichen Abhängigkeiten (nqptp, libplist, libsodium, libavahi-client) - zu fehleranfällig, um das blind ohne echte Hardware zum Gegenprüfen zu skripten.

Wichtig: Noch NICHT an echter Hardware verifiziert - weder der apt install shairport-sync-Ablauf noch das tatsächliche Ducking-Verhalten (Pause/Fortsetzen im richtigen Moment, kein hörbares Überlappen) wurden bisher an einem echten Gerät gegengeprüft, nur die OwlBox-eigene Seite (Engine-Methoden, API-Endpunkte, Kiosk-Abzeichen) über die Testsuite bzw. den Simulationsmodus.

5. RC522 RFID-Leser (Hardware-SPI0)

Hinweis: Der RC522 hängt an SPI0, dem Hardware-SPI-Bus des Pi (/dev/spidev0.0, CE0). Grund, warum das möglich ist: SPI1 liegt auf GPIO18-21, exakt den Pins, die der HiFiBerry für I2S-Ton braucht - SPI0 ist mit dem aktuellen DSI-Touch-Display aber frei (anders als beim früheren SPI-Display, dessen Overlay beide Chip-Selects von SPI0 belegte).

RC522-Pin	Raspberry Pi	Config-Feld
VCC	3.3V (nicht 5V!)	-
GND	GND	-
RST	GPIO26	rfid.reset_pin
SDA (CS)	GPIO8 (SPI0 CE0)	-
SCK	GPIO11 (SPI0 SCLK)	-
MOSI	GPIO10 (SPI0 MOSI)	-
MISO	GPIO9 (SPI0 MISO)	-
IRQ	nicht verbunden	-

Die mfrc522-Python-Bibliothek spricht den Bus direkt über spidev an, kein Bit-Banging mehr nötig. SPI selbst muss aktiviert bleiben (macht scripts/install.sh bereits via raspi-config nonint do_spi 0, alternativ sudo raspi-config → Interface Options → SPI).

Hinweis: Historischer Hintergrund zum Lautstärke-Encoder auf GPIO1 statt GPIO17: Das frühere SPI-Display beanspruchte zusätzlich zu SPI0/CE0/CE1 auch GPIO17 als Interrupt-Pin („pendown“) für den (nie verdrahteten) Touch-Controller - fest im damaligen Overlay einprogrammiert, unabhängig davon, ob Touch physisch angeschlossen war. Deshalb liegt der Lautstärke-Encoder-CLK auf GPIO1 (ID_SC, konventionell für ein HAT-ID-EEPROM reserviert, hier aber echt frei, da der HiFiBerry ohnehin per manueller dtoverlay-Zeile statt EEPROM-Erkennung konfiguriert wird). Mit dem neuen DSI-Display ist GPIO17 jetzt wieder frei - die Verkabelung bleibt hier trotzdem auf GPIO1, um nicht ohne Grund vom dokumentierten Standard abzuweichen; wer umverkabeln will, kann gpio.encoder_clk in config.yaml frei auf GPIO17 umstellen.

An echter Hardware bestätigt: Der RC522-Reset-Pin läuft über lgpio (owlbox/rfid/lgpio_compat.py), nicht über RPi.GPIO - obwohl die mfrc522-Bibliothek intern eigentlich fest auf RPi.GPIO setzt (wird per unittest.mock.patch umgeleitet, nur für diesen einen Pin - die SPI-Datenleitungen selbst laufen über den Kernel-spidev-Treiber). Grund: gpiozero (Taster/Encoder) braucht auf aktuellen Kerneln zwingend lgpio, weil RPi.GPIOs eigene Kantenerkennung dort mit „Failed to add edge detection“ abbricht - RPi.GPIO zeigte in diesem Prozess außerdem selbst für unbenutzte Pins sofort „already in use“-Warnungen. Deshalb läuft die komplette GPIO-Ansteuerung dieses Projekts konsistent über lgpio, nirgends mehr über RPi.GPIO.

6. Taster (vor/zurück)

Als Taster kommen Cherry-MX-Switches (3-Pin-Variante) zum Einsatz - elektrisch ganz normale Momentary-Schalter (schließt nur beim Drücken, öffnet sonst).

- Von den 3 Pins sind nur die beiden Metall-Pins die elektrischen Kontakte (diagonal gegenüberliegend); der dritte, meist aus Kunststoff, ist nur ein mechanischer Halteclip ohne elektrische Funktion und muss nirgends angeschlossen werden.
- Jeweils einer der beiden Metall-Pins an GPIO, der andere an GND. Kein externer Widerstand nötig, der interne Pull-up wird von gpiozero aktiviert.
- Da Cherry-MX-Switches (anders als billige Blechtaster) sehr sauber prellen, reicht die Standard-Entprellzeit (gpio.bounce_time, 50ms) komfortabel aus.

Funktion	BCM-Pin
Zurück	6
Weiter	5

Kurz drücken springt zum vorherigen/nächsten Track. Gedrückt halten (länger als gpio.seek_hold_seconds, Standard 0,4s) spult stattdessen im aktuellen Track vor/zurück, in Schritten von gpio.seek_step_seconds (Standard 10s) - kein Trackwechsel, solange gehalten wird.

7. Dreh-Encoder mit Taster (KY-040)

Encoder-Pin	Raspberry Pi
CLK	GPIO1 (nicht 17, siehe Kapitel 5)
DT	GPIO27
SW	GPIO22
+	3.3V
GND	GND

Das KY-040-Modul bringt eigene Pull-up-Widerstände für CLK/DT mit; die zusätzlich von gpiozero aktivierten Pull-ups des Pi stören dabei nicht (einfach parallel). Für den Taster (SW) hat das Modul in der Regel keinen eigenen Pull-up - das übernimmt gpiozero.Button(pull_up=True) im Code, hier also nichts weiter nötig.

Drehen ändert die Lautstärke (Schrittweite audio.volume_step), Drücken schaltet Play/Pause um. Ein langer Druck (gpio.shutdown_hold_seconds, Standard 4s) fährt den Pi sicher herunter - praktisch für ein Kindergerät ohne Zugriff auf ein Terminal. Auf 0 setzen, um das abzuschalten.

Zweiter Dreh-Encoder für Helligkeit (KY-040)

Gehört zum Standardaufbau - dasselbe KY-040-Modul wie beim Lautstärke-Encoder, diesmal **mit** Taster (anders als früher - der Taster wird jetzt für den Nachtmodus gebraucht, s.u.).

Encoder-Pin	Raspberry Pi
CLK	GPIO23
DT	GPIO12
SW	GPIO17 (gpio.brightness_encoder_switch)
+	3.3V
GND	GND

Drehen ändert die Helligkeit (Schrittweite gpio.brightness_step, Standard 5%), sofort und rein manuell - es gibt kein automatisches Dimmen. Diese Box nutzt bewusst nicht die eigene Touch-Helligkeitsregelung, die das Waveshare-Display selbst laut Auftraggeber mitbringt - ausschließlich Encoder und Web-UI.

Hinweis: An echter Hardware bestätigt: eine Helligkeitsänderung per Encoder/Web-UI dimmt tatsächlich sichtbar das Display - Details zur Sysfs-Backlight-Schnittstelle siehe Kapitel 8.

Nachtmodus: Ein Druck auf den Taster (SW) schaltet zwischen der normalen ("Tag"-)Helligkeit und einer separat konfigurierbaren, oft deutlich dunkleren Nachtmodus-Helligkeit um - konfigurierbar in Einstellungen → Anzeige (night_brightness, Standard 5%). Anders als die normale Helligkeit darf die Nachtmodus-Helligkeit bewusst unter die dort eingestellte Minimal-Helligkeit gehen, das ist der ganze Zweck. Kein Zeitplan - bleibt aktiv, bis erneut gedrückt wird, außer über einen Neustart hinweg (startet immer im Tag-Modus). Ändert sich die Helligkeit während des Nachtmodus direkt (Schieberegler, Drehen an demselben Encoder), beendet das den Nachtmodus automatisch, statt den neuen Wert beim nächsten Tastendruck

wieder zu verwerfen. gpio.brightness_encoder_switch auf null setzen, um den Taster unverdrahtet zu lassen und die Funktion nur über die Web-UI nutzbar zu machen.

Hinweis: Damit Shutdown/Neustart (auch über die Web-UI oder einen Funktions-Chip) sowie der Update-Button auf der Info-Seite ohne Passwortabfrage funktionieren, braucht der Service-User owlbox passwortloses sudo dafür. install.sh richtet das automatisch ein (/etc/sudoers.d/owlbox, syntaxgeprüft vor dem Einspielen) - hier nur zur Referenz:

```
owlbox ALL=(ALL) NOPASSWD: /sbin/shutdown, /usr/bin/nmcli, \
/usr/bin/systemctl restart --no-block owlbox
```

Wichtig: Die Argumente müssen exakt wie oben dastehen (inklusive --no-block) - sudo vergleicht die komplette Befehlszeile. Fehlt --no-block, meldet der Update-Button auf der Info-Seite beim Neustart „sudo: a password is required“.

8. Display-Hintergrundbeleuchtung

Hinweis: An echter Hardware bestätigt: dieses Display bietet Helligkeitsregelung über eine interne Linux-Backlight-Sysfs-Schnittstelle an (/sys/class/backlight//brightness, auf dem geprüften Gerät als /sys/class/backlight/11-0045/ zu finden - die Zahl davor ist eine I2C-Bus/Adress-Kombination und kann je nach Board/Kernel-Version abweichen; max_brightness war 255, die Datei group-schreibbar für video, dieselbe Gruppe, die owlbox.service ohnehin schon hat). owlbox/backlight.py erkennt dieses Sysfs-Gerät automatisch und nutzt es - keine GPIO-Verkabelung nötig, passend dazu, dass die Anschluss-Tabelle oben nur 4 Kabel (5V/GND/SDA/SCL) kennt. Der alte GPIO13-PWM-Ansatz bleibt als Fallback im Code, greift aber nur noch, wenn kein Sysfs-Gerät gefunden wird.

Software-seitig ist der zweite Encoder vollständig verdrahtet und angebunden (CLK/DT fürs Drehen, SW für den Nachtmodus-Taster, siehe Kapitel 7) - jede Helligkeitsänderung über Encoder, Web-Oberfläche oder Nachtmodus-Taster ändert den intern gespeicherten brightness-Wert zuverlässig, blendet ihn kurz auf dem Display ein und dimmt jetzt auch tatsächlich die Display-Helligkeit sichtbar.

9. Fallback-Hotspot (WLAN-Recovery)

Ist WLAN eingeschaltet, aber für network.hotspot_after_seconds (Standard 60s) mit keinem Netzwerk verbunden - z.B. weil das Heimnetz sein Passwort geändert hat oder der Pi an einen neuen Ort umgezogen ist - macht der Pi automatisch seinen eigenen Access Point auf (nmcli device wifi hotspot), statt komplett unerreichbar zu bleiben.

SSID und Passwort (aus config.yaml unter network:, Standard „OwlBox-Setup“ / „owlbox-setup“) werden auf dem Kiosk-Display und im Admin-Bereich angezeigt, inklusive der URL, unter der die Einstellungen im Hotspot erreichbar sind (normalerweise http://10.42.0.1:5000/admin - NetworkManagers Standard-Adresse für einen geteilten Access Point).

Ablauf: mit einem Laptop/Handy in dieses WLAN einwählen, die angezeigte URL öffnen, unter Einstellungen → WLAN das eigentliche Netzwerk auswählen/verbinden. Sobald das klappt, beendet der Pi den Hotspot von selbst wieder.

Wichtig: Das Standardpasswort „owlbox-setup“ steht so im Quellcode und ist damit öffentlich bekannt - für den Einsatz in einer Umgebung, in der Fremde in Funkreichweite kommen könnten, unbedingt in config.yaml ein eigenes Passwort setzen.

10. DSI-Display: kein separater Treiber-Installer nötig

Im Gegensatz zum früheren 3,5″-SPI-Display (das einen virtuellen-HDMI-Trick, fbcp und den alten Legacy-Grafiktreiber brauchte, um überhaupt ein Bild zu zeigen) kommt das aktuelle DSI-Display an einem normalen Raspberry Pi OS Bookworm-Image ohne Treiber-Installer, ohne Extra-Paket aus - nur die eine dtoverlay=-Zeile unten ist nötig, sonst nichts. Empfohlenes Basis-Image bleibt Raspberry Pi OS Lite, 64-bit (ohne Desktop-Umgebung) - der Kiosk startet X selbst nur für Chromium (siehe Kapitel 11), eine mitinstallierte Desktop-Umgebung (lightdm, LXDE) würde beim Boot nur unnötig Zeit kosten. Wichtig ist nur: der moderne KMS-Grafiktreiber (vc4-kms-v3d) bleibt aktiv (Bookworm-Standard) - er wurde beim alten SPI-Display extra deaktiviert, das ist mit einem DSI-Display nicht mehr nötig und würde die GPU-Beschleunigung sogar wieder kosten.

Wichtig: Noch NICHT an echter Hardware verifiziert. Laut Waveshare-Wiki (waveshare.com/wiki/5-DSI-TOUCH-A): dtoverlay=vc4-kms-v3d,noaudio allein aktiviert nur den generellen KMS-Treiber, kennt aber die Timings dieses konkreten Panels nicht - zusätzlich braucht es dtoverlay=vc4-kms-dsi-waveshare-panel-v2,5_0_inch_a (NICHT das bisherige dtoverlay=vc4-kms-dsi-7inch, das war spezifisch für das alte offizielle Display). Beim alten Display äußerte sich ein fehlender displayspezifischer Overlay als komplett schwarzer Bildschirm (dmesg: „[drm] Cannot find any crtc or sizes“) - für dieses Display noch nicht bestätigt, aber vermutlich vergleichbar. install.sh trägt beide Zeilen automatisch in /boot/firmware/config.txt ein:

```
dtparam=i2c_arm=on
dtoverlay=vc4-kms-v3d,noaudio
dtoverlay=vc4-kms-dsi-waveshare-panel-v2,5_0_inch_a
```

Drehung/Ausrichtung: bewusst nicht konfiguriert

Das Panel ist nativ 720×1280 (Hochformat) - anders als das bisherige, physisch auf dem Kopf verbaute 800×480-Display wird die Ausrichtung hier über den physischen Einbau gelöst, nicht per Software. Falls sich das nach dem Einbau doch als nötig herausstellt, zwei Dinge, die beim alten Display an echter Hardware mühsam herausgefunden wurden und vermutlich weiter gelten:

- **Bild selbst:** gehört als Kernel-Boot-Parameter in cmdline.txt (nicht config.txt!): video=DSI-1:<Modus>,rotate=<Grad>. NICHT über xrandr oder display_lcd_rotate versuchen - an echter Hardware bestätigt (beim alten Display): xrandr --output DSI-1 --rotate wird zwar anstandslos angenommen, das Panel zeichnet aber nie tatsächlich neu, selbst nach einem erzwungenen Modeset. display_lcd_rotate/lcd_rotate ist unter KMS ebenfalls wirkungslos.
- **Touch-Koordinaten:** bei 180° hat sich beim alten Display gezeigt, dass X11/libinput die Touch-Koordinaten am gedrehten Ausgang automatisch mitkorrigiert, zusätzlich gesetztes invx/invy hat das nochmal drübergedreht. Bei 90°/270° ist das noch nicht getestet - anders als bei 180° vertauschen 90°/270° die X/Y-Achsen, ob das automatisch mitkorrigiert wird oder invx/invy/swapxy von Hand nötig sind, muss an echter Hardware geprüft werden.

11. Kiosk-Autostart (Chromium fullscreen, ohne Desktop-Umgebung)

Da die Basis „Lite“ keine Desktop-Umgebung mitbringt, gibt es auch kein lightdm/LXDE, in das sich der Kiosk einhängen könnte. Stattdessen startet ein eigener systemd-Dienst (owlbox-kiosk.service) X direkt selbst (per startx), übernimmt dafür tty1 und lässt scripts/kiosk.sh (das Chromium im Kiosk-Modus startet) als einzigen „Client“ laufen - keine Fensterleiste, kein Dateimanager-Desktop, kein Panel. Mit aktivem KMS-Treiber findet X's eigener, automatisch gewählter „modesetting“-Treiber /dev/dri/card0 von selbst - keine eigene Xorg-Konfiguration nötig (anders als beim alten Display, das X explizit auf einen Framebuffer-Treiber zwingen musste).

```
# X ohne Display-Manager erlauben:
cat > /etc/X11/Xwrapper.config <<'EOF'
allowed_users=anybody
needs_root_rights=yes
EOF

sudo cp /opt/owlbox/systemd/owlbox-kiosk.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now owlbox-kiosk.service
```

Hinweis: owlbox-kiosk.service bringt Conflicts=getty@tty1.service schon mit, muss also getty@tty1.service nicht extra deaktiviert bekommen - läuft aber sauberer, wenn man es trotzdem tut (sudo systemctl disable getty@tty1.service), damit dort kein ungenutzter Login-Prompt mehr mitstartet.

Läuft doch eine volle Desktop-Umgebung (z.B. weil bewusst die volle Variante statt Lite geflasht wurde), lässt sich der Kiosk alternativ ganz klassisch über deren Autostart-Datei einhängen: ~/.config/lxsession/LXDE-pi/autostart um die Zeile @/opt/owlbox/scripts/kiosk.sh ergänzen.

Hinweis: owlbox-kiosk.service läuft bewusst mit niedrigerer CPU-/IO-Priorität als owlbox.service (Nice=15, IOSchedulingClass=best-effort, IOSchedulingPriority=7). Grund: Chromium läuft auf dieser Hardware komplett softwaregerendert (keine GPU-Beschleunigung verfügbar) - ohne Prioritätsdifferenz konkurrieren Chromium und mpv (läuft in owlbox.service) mit exakt gleicher Priorität um die knappe CPU eines Pi 3B+. Ein positiver Nice-Wert braucht keine besonderen Rechte - install.sh trägt das automatisch ein und startet owlbox-kiosk.service bei Bedarf neu, damit die neue Priorität auch ohne kompletten Neustart greift.

Wichtig: Hinweis für den Pi 5: Der komplette Rest dieses Abschnitts (Nice-Fix, die folgenden Korrekturen/Vermutungen zum Knacken, die entfernten Dauerschleifen) wurde ausschließlich an einem Pi 3B+ untersucht - "die knappe CPU eines Pi 3B+" trifft auf einen Pi 5 (deutlich schnellere CPU, zudem echte GPU-Beschleunigung für Chromium grundsätzlich möglich) so womöglich gar nicht mehr zu. Die Priorisierung selbst bleibt harmlos, ob sie auf einem Pi 5 überhaupt noch etwas bewirkt ist offen - noch nicht an echter Pi-5-Hardware verifiziert.

Wichtig: Korrektur, an echter Hardware geprüft: Dieser Nice-Fix allein hat ein durchgehendes Knacken bei Wiedergabe NICHT behoben - mit dem Fix aktiv knackte es an echter Hardware weiterhin. vcgencmd get_throttled zeigte 0x0 (keine Unterspannung/Drosselung), top zeigte im knackenden Zustand noch 62.5% CPU im Leerlauf (Load Average 0.52) und dmesg keinerlei ALSA-/I2S-Fehler - die CPU war im klassischen Sinn nie wirklich ausgelastet. Die Priorisierung bleibt trotzdem bestehen (kostet nichts), war aber nicht die vollständige Lösung. Aktuelle, noch nicht an echter Hardware verifizierte Vermutung: kontinuierliches Repaint im Kiosk selbst (unendlich laufende CSS-Animation für lange Story-Titel als Laufschrift, sowie ein setInterval alle 450ms für die VU-Meter-Balken) erzeugte auf dem softwaregerenderten Chromium regelmäßige kurze Lastspitzen, die in einer top-Momentaufnahme nicht auffallen, aber mpvs Audio-Thread gelegentlich einen Scheduling-Slot gekostet haben könnten. Beide Dauerschleifen wurden entfernt (Laufschrift durch einfaches Abschneiden mit "..." ersetzt, VU-Balken randomisieren nur noch einmal beim Start der Wiedergabe) - dieser Fix ist Stand jetzt noch nicht an echter Hardware getestet.

12. Konfigurationsdatei (config.yaml)

Alle in dieser Anleitung genannten Werte (Pins, Schrittweiten, Zeiten) stehen gesammelt in config/config.yaml (aus config/config.example.yaml kopieren). Die wichtigsten Abschnitte:

Abschnitt	Wichtigste Werte
audio:	alsa_device, mixer_control, default_volume, volume_step, chime_enabled
rfid:	reader, reset_pin, poll_interval
gpio:	button_next/prev, encoder_clk/dt/switch, backlight_pin, brightness_encoder_clk/dt, shutdown_hold_seconds, seek_hold_seconds
playback:	restart_track_after_seconds, position_save_interval, auto_sleep_minutes, sleep_fade_seconds
network:	hotspot_ssid, hotspot_password, hotspot_after_seconds
web:	host, port, secret_key

Hinweis: config/config.yaml ist in .gitignore eingetragen, damit lokale, gerätespezifische Werte (insb. das Session-Secret) nie versehentlich committet werden.